

# ESTRUCTURAS DE DATOS

## Práctica de Laboratorio N° 08: Implementación de Árboles Binarios

### I. OBJETIVOS DE LA PRÁCTICA:

- Comprender las características esenciales del árbol binario como una estructura de datos dinámica, no lineal y recursiva, capaz de representar relaciones jerárquicas entre elementos de forma eficiente.
- Desarrollar la implementación de un árbol binario en C++, utilizando memoria dinámica para construir nodos e insertar datos de forma interactiva.
- Aplicar funciones recursivas para operar sobre el árbol, incluyendo los recorridos en preorden, inorden y posorden.

### II. DESARROLLO DE LA PRÁCTICA:

1. Un **árbol binario** es una estructura de datos **dinámica, no lineal y recursiva**, en la cual cada nodo puede tener a lo sumo dos hijos: uno izquierdo y uno derecho. Es útil para representar relaciones jerárquicas, optimizar búsquedas y organizar información. En esta práctica se implementará un árbol binario utilizando nodos creados dinámicamente (punteros). Se creará una clase llamada Arbol y se desarrollarán los siguientes métodos:

- **creaArbol()** – Crea el árbol binario de forma recursiva, permitiendo al usuario ingresar los valores de cada nodo y decidir si existen hijos izquierdo y derecho.
- **muestraArbol()** – Muestra visualmente la estructura del árbol en consola, alineando los niveles jerárquicamente.
- **preorden()** – Recorre el árbol en preorden (raíz – izquierda – derecha) y muestra los valores.
- **inorden()** – Recorre el árbol en inorden (izquierda – raíz – derecha) y muestra los valores.
- **posorden()** – Recorre el árbol en posorden (izquierda – derecha – raíz) y muestra los valores.
- **altura()** – Calcula y devuelve la altura del árbol (la longitud del camino más largo desde la raíz hasta una hoja).
- **contarNodos()** – Cuenta el número total de nodos existentes en el árbol.

Cada uno de estos métodos utilizará **recursividad** para facilitar el procesamiento descendente de los nodos, reflejando así la naturaleza estructural del árbol binario.

```
1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  class Nodo
6  {
7  public:
8      int info;
9      Nodo* izq;
10     Nodo* der;
11 };
12
13 class Arbol
14 {
15 private:
16     Nodo* raiz; //apuntador al inicio del arbol
17 }
```

# ESTRUCTURAS DE DATOS

```
18 public:
19     Arbol();    //constructor
20
21     Nodo* creaArbol();
22     void muestraArbol(Nodo *apnodo, int nivel);
23
24     // Métodos de recorrido
25     void preorden(Nodo* apnodo);
26     void inorden(Nodo* apnodo);
27     void posorden(Nodo* apnodo);
28
29     int altura(Nodo* apnodo);
30     int contarNodos(Nodo* apnodo);
31     int contarHojas(Nodo* apnodo);
32 };
33
34 //constructor predeterminado
35 Arbol::Arbol()
36 {
37     raiz = NULL;
38 }
39
40 void Arbol::muestraArbol(Nodo *apnodo, int nivel)
41 {
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
```

```
18 public:
19     Arbol();    //constructor
20
21     Nodo* creaArbol();
22     void muestraArbol(Nodo *apnodo, int nivel);
23
24     // Métodos de recorrido
25     void preorden(Nodo* apnodo);
26     void inorden(Nodo* apnodo);
27     void posorden(Nodo* apnodo);
28
29     int altura(Nodo* apnodo);
30     int contarNodos(Nodo* apnodo);
31     int contarHojas(Nodo* apnodo);
32 };
33
34 //constructor predeterminado
35 Arbol::Arbol()
36 {
37     raiz = NULL;
38 }
39
40 void Arbol::muestraArbol(Nodo *apnodo, int nivel)
41 {
42
43
44
45
46
47
48
49
50
51
52
53
54
55     Nodo* Arbol::creaArbol()
56     {
57         bool resp;
58         Nodo* apnodo = new Nodo();
59
60         cout << "\nIngrese el valor del nodo: ";
61         cin >> apnodo->info;
62
63         cout << "\n ¿Existe nodo por izquierda: 1 (Sí) - 0 (No)? ";
64         cin >> resp;
65         if(resp == 1){
66             apnodo->izq = creaArbol();
67         }else{
68             apnodo->izq = NULL;
69         }
70
71         cout << "\n ¿Existe nodo por derecha: 1 (Sí) - 0 (No)? ";
72         cin >> resp;
73         if(resp == 1){
74             apnodo->der = creaArbol();
75         }else{
76             apnodo->der = NULL;
77         }
78
79         return apnodo;
80     }
81
```

# ESTRUCTURAS DE DATOS

```
82 // Recorrido en Preorden: Raiz - Subárbol izquierdo - Subárbol Derecho
83 void Arbol::preorden(Nodo* apnodo)
84 {
85     if (apnodo != NULL) {
86         cout << apnodo->info << " - ";
87         preorden(apnodo->izq);
88         preorden(apnodo->der);
89     }
90 }
91
92 // Recorrido en Inorden: Subárbol izquierdo - Raiz - Subárbol Derecho
93 void Arbol::inorden(Nodo* apnodo)
94 {
95 }
96
97 // Recorrido en Posorden: Subárbol izquierdo - Subárbol Derecho - Raiz
98 void Arbol::posorden(Nodo* apnodo)
99 {
100 }
101
102 // Altura del árbol
103 int Arbol::altura(Nodo* apnodo)
104 {
105     if (apnodo == NULL) {
106         return 0;
107     } else {
108         return 1 + max(altura(apnodo->izq), altura(apnodo->der));
109     }
110 }
111
112 int Arbol::contarNodos(Nodo* apnodo)
113 {
114     if (apnodo == NULL) {
115         return 0;
116     } else {
117         return 1 + contarNodos(apnodo->izq) + contarNodos(apnodo->der);
118     }
119 }
120
121 int Arbol::contarHojas(Nodo* apnodo)
122 {
123 }
124
125 void menu()
126 {
127     Arbol arbol;
128     Nodo* raiz = NULL;
129     int opcion;
130
131     do {
132         cout << "\n==== MENÚ DE OPCIONES - ÁRBOL BINARIO =====> endl;
133         cout << "1. Crear árbol" << endl;
134         cout << "2. Mostrar árbol (forma estructurada)" << endl;
135         cout << "3. Recorrido en Preorden" << endl;
136         cout << "4. Recorrido en Inorden" << endl;
137         cout << "5. Recorrido en Posorden" << endl;
138         cout << "6. Altura del árbol" << endl;
139         cout << "7. Contar todos los nodos" << endl;
140         cout << "8. Contar nodos hoja" << endl;
141         cout << "0. Salir" << endl;
142         cout << "Seleccione una opción: ";
143         cin >> opcion;
144     } while (opcion != 0);
145 }
```

# ESTRUCTURAS DE DATOS

```
165 switch(opcion) {
166     case 1:
167         raiz = arbol.creaArbol();
168         break;
169     case 2:
170         if (raiz == NULL)
171             cout << "El árbol está vacío.\n";
172         else
173             arbol.muestraArbol(raiz, 0);
174         break;
175     case 3:
176         if (raiz == NULL) {
177             cout << "El árbol está vacío.\n";
178         } else {
179             cout << "Recorrido Preorden : ";
180             arbol.preorden(raiz);
181             cout << endl;
182         }
183         break;
184     case 4:
185         if (raiz == NULL) {
186             cout << "El árbol está vacío.\n";
187         } else {
188             cout << "Recorrido Inorden : ";
189             arbol.inorden(raiz);
190             cout << endl;
191         }
192         break;
193     case 5:
194         if (raiz == NULL) {
195             cout << "El árbol está vacío.\n";
196         } else {
197             cout << "Recorrido Posorden : ";
198             arbol.posorden(raiz);
199             cout << endl;
200         }
201         break;
202     case 6:
203         if (raiz == NULL)
204             cout << "El árbol está vacío.\n";
205         else
206             cout << "Altura del árbol: " << arbol.altura(raiz) << endl;
207         break;
208     case 7:
209         if (raiz == NULL)
210             cout << "El árbol está vacío.\n";
211         else
212             cout << "Cantidad total de nodos: " << arbol.contarNodos(raiz) << endl;
213         break;
214     case 8:
215         if (raiz == NULL)
216             cout << "El árbol está vacío.\n";
217         else
218             cout << "Cantidad de nodos hoja: " << arbol.contarHojas(raiz) << endl;
219         break;
220     case 0:
221         cout << "Saliendo del programa...\n";
222         break;
223     default:
224         cout << "Opción inválida. Intente nuevamente.\n";
225 }
226 } while(opcion != 0);
227 }
228 }
```

# ESTRUCTURAS DE DATOS

```
230 int main()
231 {
232     setlocale(LC_ALL, "");
233
234     menu();
235
236     /*
237     Arbol arbol;
238     Nodo* raiz = arbol.creaArbol();
239     cout << "\nRepresentación del árbol:\n";
240     arbol.muestraArbol(raiz, 0);
241
242     cout << "\nRecorrido Preorden : ";
243     arbol.preorden(raiz);
244
245     cout << "\nRecorrido Inorden : ";
246     arbol.inorden(raiz);
247
248     cout << "\nRecorrido Posorden : ";
249     arbol.posorden(raiz);
250
251     cout << "\n\nAltura del árbol: " << arbol.altura(raiz) << endl;
252
253     cout << "\nCantidad total de nodos en el árbol: " << arbol.contarNodos(raiz) << endl;
254
255     int hojas = arbol.contarHojas(raiz);
256     cout << "\nCantidad de nodos hoja: " << hojas << endl;
257
258     */
259
260     return 0;
261 }
```

2. Implemente un método que muestre en pantalla el árbol en forma horizontal, con la raíz a la izquierda.
3. Escriba una función recursiva que cuente cuántos nodos hoja tiene un árbol binario. Además, la función debe mostrar por pantalla el valor de cada nodo hoja encontrado.

**Ejemplos de salida:**

Nodo hoja encontrado: 5  
Nodo hoja encontrado: 12  
Nodo hoja encontrado: 27  
Cantidad de nodos hoja: 3